The Journal of Open Source Software

StructuralVibration.jl: A Julia package for structural vibration analysis

Mathieu Aucejo [1]

1 Laboratoire de Mécanique des Structures et des Systèmes Couplés, Conservatoire National des Arts et Métiers, Paris, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 19 March 2026
Published: unpublished

Summary

`StructuralVibration.jl` is a Julia package designed to generate and analyze vibration data for mechanical systems. It provides tools for modeling, simulating, and analyzing the dynamic behavior of mechanical structures. This package can be used for educational and research purposes. Users can create models of mechanical systems, simulate their response to various inputs, and analyze the resulting vibration data. The package also includes features for visualizing vibration data in 2D and 3D, animating vibration modes and vibration fields, or performing experimental modal analysis. `StructuralVibration.jl` is built on top of the Julia programming language, which offers high performance and ease of use for scientific computing tasks.

Statement of need

Structural vibration analysis is a crucial aspect of mechanical engineering, as it helps engineers understand the dynamic behavior of structures and predict how they will respond to different inputs. It covers a wide range of applications, including the design of buildings, bridges, vehicles, and machinery. Accurate vibration analysis is essential for ensuring the safety, reliability and optimal performance of these structures.

Analyzing the dynamic behavior requires a combination of theoretical knowledge in both mechanics (Geradin & Rixen, 2015) and signal processing (Brandt, 2011), computational tools, and experimental techniques (Avitabile, 2018). Mechanical engineers must be able to model the behavior of structures under different loading conditions, simulate their responses to various inputs, and analyze the resulting vibration data. This process can be complex and time-consuming.

To the best of our knowledge, there is currently no package dedicated to structural vibration analysis available in the Julia programming language. `StructuralVibration.jl` aims to fill this gap by providing an accessible, package for generating and analyzing vibration data in Julia. It is designed to be user-friendly for beginners and experienced engineers alike, it is a valuable resource for education and research in the field of structural dynamics.

State of the field

Many tools are available for structural vibration analysis. Most of them are written in languages such as Matlab or Python. For example, the Python ecosystem for structural vibration analysis is fragmented (Slavic et al., 2025), with various libraries and tools available for different aspects of the analysis. These include `pyOMA2`, `pyFBS`, `OpenSeesPy` or `SDynPy`. In Matlab, one can cite `Structural Dynamics Toolbox` (SDTools) or the Matlab's `System Identification Toolbox`.

This can make it difficult for engineers to find the right tools for their specific needs and to integrate them into a cohesive workflow.

Software design

`StructuralVibration.jl` is designed according to certain guidelines to ensure that it is user-friendly, efficient, extensible, and composable. These guidelines include:

- *Type stability*: All of the package's functions are designed to ensure type stability, allowing for efficient execution and better performance.
- *Type genericity*: The package is designed to be type-generic, enabling users to work with various data types, such as `Float64`, `ComplexF32`, while maintaining performance and flexibility.
- *Composability*: The package is design to be compatible with other Julia packages. For example, users can define a state-space model using `StructuralVibration.jl` and perform the resolution using another package such as `DifferentialEquations.jl`.
- *Reliability*: `StructuralVibration.jl` includes a comprehensive documentation and a test suite that ensures the reliability and correctness of the package. The documentation provides detailed explanations of the theoretical background of the package's features, as well as the API of the data types and functions implemented. Various examples are also provided to help users get started.
- *Modularity*: The package is based on a modular architecture that clearly separates the different functionalities (Modelling & Simulation, Signal Processing, Modal Extraction, Visualization & Animation) and facilitates the maintenance and extension. The general architecture of the package is presented in Figure 1.

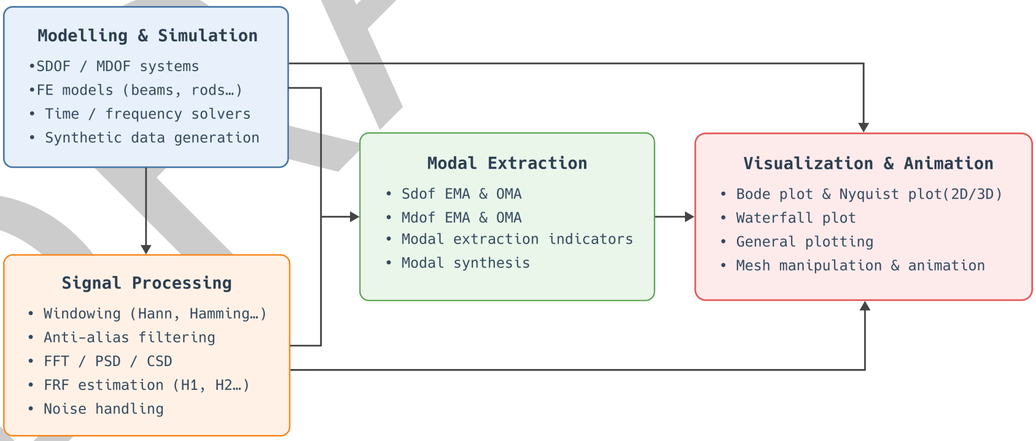

Figure 1: General architecture of StructuralVibration.jl.

Research impact statement

`StructuralVibration.jl` has already been used for educational purposes in a structural dynamics course at the Conservatoire National des Arts et Métiers (Cnam) in Paris, France. It has also been used for research purposes in the context of force reconstruction from vibration data (Aucejo & De Smet, 2025). The package is open-source under an MIT license and available on GitHub, allowing for community contributions and future development.

Example

The example presented below demonstrates how to use `StructuralVibration.jl` to create a simple model of a simply supported beam, simulate its transfer function matrix, extract its first four vibration modes using the Least-Squares Complex Frequency-domain (LSCF) method, and visualize the mode shapes (see Figure 2).

Creation of the beam model

```julia
using StructuralVibration

# Geometric and material properties
L = 1.         # Length
b = 0.03       # Width
h = 0.01       # Thickness
S = b*h        # Cross-section area
Iz = b*h^3/12 # Moment of inertia

E = 2.1e11  # Young's modulus
ρ = 7850.   # Density
ξ = 0.01    # Damping ratio

# Create the beam model
beam = Beam(L, S, Iz, E, ρ)
```

Simulation of the transfer function matrix

```julia
# Mesh definition
xexc = 0:0.05:L
xm = xexc[2]

# Vibration modes calculation
fmax = 500.
ωn, kn = modefreq(beam, 2fmax)
ms_exc = modeshape(beam, kn, xexc)
ms_m = modeshape(beam, kn, xm)

# FRF calculation
freq = 1.:0.1:fmax
prob = ModalFRFProblem(ωn, ξ, freq, ms_m, ms_exc)
H = solve(prob).u
```

Mode extraction

```julia
# EMA problem
prob_mdof = EMAProblem(H, freq)

# Stabilization diagram analysis using the LSCF method
order = 10
stab = stabilization(prob_mdof, order, LSCF())
stabilization_plot(stab)

# Compute the resonance frequencies and damping ratios from the poles
p_lscf = poles_extraction(prob_mdof, order, LSCF())
fn, ξn = poles2modal(p_lscf)
```

The Journal of Open Source Software

```julia
# Mode shapes extraction
dpi = [1, 2]
res = mode_residues(prob_mdof, p_lscf)
ms_est = modeshape_extraction(res, p_lscf, LSCF(), dpi = dpi, modetype = :emar)[1]
ms_est_real = real_normalization(ms_est)
```

Visualization of the mode shapes

```julia
using CairoMakie

set_theme!(theme_choice(:sv))

# Mode shapes visualization
id_mode = 2
sv_plot(
  xexc, ms_exc[:, id_mode], ms_est_real[:, id_mode],
  title = "Mode shape of mode $id_mode", lw = 2., xlabel = "x (m)", ylabel = "Value",
  legend = (active = true, position = :rt, entry = ("Reference", "Estimated"))
)
```

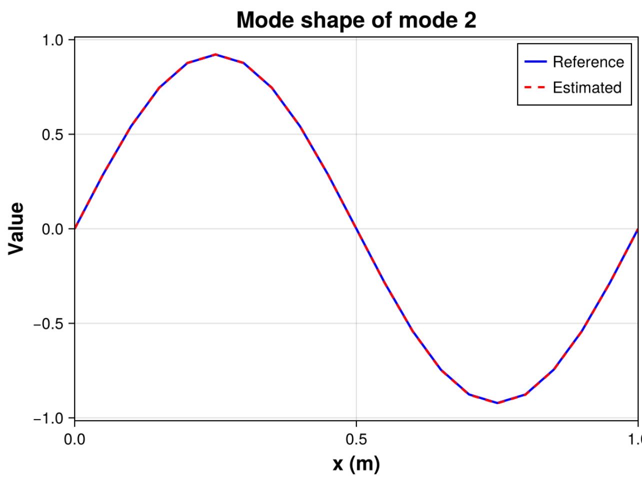

Figure 2: Visualization example - Second mode shape of the beam.

AI usage disclosure

The author of the package wrote and proofread the codes and documentation of `StructuralVibration.jl`. No generative AI was used during development.

References

Aucejo, M., & De Smet, O. (2025). A frequency-domain sequential Bayesian filter for sparse and broadband force estimation problems. *Mechanical Systems and Signal Processing*, *232*, 112729. https://doi.org/https://doi.org/10.1016/j.ymssp.2025.112729

Avitabile, P. (2018). *Modal Testing: A practitioner's guide*. Wiley.

Brandt, A. (2011). *Noise and Vibration Analysis: Signal analysis and experimental procedures*. Wiley. https://onlinelibrary.wiley.com/doi/book/10.1002/9780470978160

Geradin, M., & Rixen, D. (2015). *Mechanical vibrations: Theory and application to structural dynamics*. Wiley. https://www.wiley.com/en-us/Mechanical+Vibrations%3A+Theory+and+Application+to+Structural+Dynamics%2C+3rd+Edition-p-9781118900208

Slavic, J., Zaletelj, K., & Gorjup, D. et al. (2025). Python open-source signal processing and modeling/simulation for scientific research in structural dynamics—A review. *Mechanical Systems and Signal Processing*, *241*, 113465. https://doi.org/10.1016/j.ymssp.2025.113465